Part A – Application area review - Market Analysis

1.**Introduction**

Market analysis is a major application area of artificial intelligence because economic markets produce large amounts of time-series data. These data include consumer prices, exchange rates, interest rates, money supply, commodity prices, fuel prices, and demand indicators. In this project, market analysis is explored through Consumer Price Index (CPI) forecasting. CPI is a key measure of inflation and reflects changes in the average price of goods and services paid by consumers [26]. Accurate CPI forecasting is important for policymakers, central banks, businesses, and households because inflation affects purchasing power, investment decisions, wage planning, and monetary policy.

2.**Traditional Approaches in CPI Forecasting**

Before the growth of artificial intelligence, CPI and inflation forecasting mainly depended on statistical and econometric models. ARIMA is one of the most widely used models for time-series forecasting. The Box-Jenkins method provides a structured way to model historical values through autoregressive, differencing, and moving average components [19]. ARIMA has been used in many developing-country contexts because it is simple, interpretable, and suitable for short-term forecasting. For example, Faisal [14] applied ARIMA to forecast inflation in Bangladesh, while Akhter [39] used seasonal ARIMA to forecast monthly CPI data in Bangladesh. These studies show that ARIMA and SARIMA remain useful benchmark models for inflation forecasting.

3.**Use of AI and Machine Learning**

Although ARIMA is useful, inflation is often affected by nonlinear and unexpected factors such as exchange rate depreciation, fuel price shocks, food price volatility, global commodity prices, and policy uncertainty. Because of this, machine learning techniques are increasingly used in market analysis. Machine learning models can process larger datasets and identify complex relationships among many predictors. Liu, Pan, and Xu [1] argued that machine learning can improve inflation forecasting by using wider economic datasets and focusing on out-of-sample performance. Medeiros et al. [2] also found that machine learning methods can improve inflation forecasting in data-rich environments.

Tree-based machine learning models such as Random Forest and XGBoost have also been applied in inflation forecasting. Random Forest combines multiple decision trees to improve prediction and reduce variance [42]. XGBoost is a scalable boosting method that improves accuracy by correcting previous errors in sequence [9]. Li et al. [4] used XGBoost with genetic algorithm optimisation and found that it improved multi-horizon inflation forecasting.

4.**Deep Learning and Hybrid Models**

Deep learning models are also important for CPI forecasting, especially when the data contain nonlinear patterns and long-term dependencies. Long Short-Term Memory, or LSTM, is a recurrent neural network model designed for sequential data [23]. Studies have shown that LSTM can perform well in inflation forecasting when price movements are unstable or nonlinear [24], [25]. Siami Namini et al. [34] compared ARIMA and LSTM and found that LSTM often produced lower forecasting errors. Hybrid models are also becoming popular. For example, Peirano et al. [18] used a SARIMA-LSTM model for Latin American inflation forecasting, while Gur [16] found that a hybrid LSTM-XGBoost model performed strongly for CPI forecasting.

5.**Relevance to This Project**

For Bangladesh, CPI forecasting is highly relevant because inflation is influenced by food prices, imported fuel, exchange rate pressure, and global commodity shocks. Existing studies in Bangladesh have used ARIMA and machine learning models, but more comparison is needed between traditional and modern AI-based models [10], [14], [39]. Therefore, this project focuses on CPI forecasting as a market analysis problem and compares classical time-series methods with machine learning or deep learning approaches.

6.**Statement on the Use of ChatGPT**

ChatGPT was used to assist with structuring and improving the academic language of this literature review. It was not used to collect or manipulate data. The project topic, references, and final academic responsibility remain with the student.

Referrence

[1] Y. Liu, R. Pan, and R. Xu, "Mending the crystal ball: Enhanced inflation forecasts with machine learning," IMF Working Paper WP/24/206, 2024.

[2] M. C. Medeiros, G. F. R. Vasconcelos, A. Veiga, and E. Zilberman, "Forecasting inflation in a data rich environment: The benefits of machine learning methods,"

Journal of Business and Economic Statistics, vol. 39, no. 1, pp. 98 to 119, 2021.

[4] S. Li et al., "Forecasting inflation rates by extreme gradient boosting with the genetic algorithm,"* Journal of Ambient Intelligence and Humanized Computing,* 2022.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785 to 794.

[10] L. B. Ismail, M. I. Joytu, T. I. Plabon, and M. S. Oshman, "Evaluation of machine learning models to forecast inflation: Bangladesh as a case study," Proceedings of the International Symposium on Networks, Computers and Communications, 2023.

[14] M. Faisal, "Forecasting Bangladesh's inflation using time series ARIMA models," World Review of Business Research, vol. 2, no. 3, pp. 100 to 117, 2012.

[16] Y. E. Gur, "Development and application of machine learning models in US consumer price index forecasting: Analysis of a hybrid approach," Data Science in Finance and Economics, vol. 4, no. 4, pp. 469 to 513, 2024.

[18] R. Peirano, W. Kristjanpoller, and M. C. Minutolo, "Forecasting inflation in Latin American countries using a SARIMA LSTM combination," Soft Computing, vol. 25, no. 16, pp. 10851 to 10862, 2021.

[19] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden Day, 1976.

[23] S. Hochreiter and J. Schmidhuber, "Long short term memory," Neural Computation, vol. 9, no. 8, pp. 1735 to 1780, 1997.

[24] A. Almosova and N. Andresen, "Nonlinear inflation forecasting with recurrent neural networks," Journal of Forecasting, vol. 42, no. 2, pp. 240 to 259, 2023.

[25] L. Paranhos, "Predicting inflation with recurrent neural networks," arXiv preprint arXiv:2104.03757, 2021.

[26] International Monetary Fund, "Inflation: Prices on the Rise," IMF Finance and Development, 2013.

[34] S. Siami Namini, N. Tavakoli, and A. S. Namin, "A comparison of ARIMA and LSTM in forecasting time series," Proceedings of the IEEE International Conference on Machine Learning and Applications, 2018.

[39] T. Akhter, "Short term forecasting of inflation in Bangladesh with seasonal ARIMA processes," MPRA Paper No. 43729, 2013.

[42] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5 to 32, 2001.

Part B: Compare and Evaluate AI

Goal of the Application

The goal of this project is to forecast future Consumer Price Index (CPI) values using historical CPI data. The dataset contains annual CPI information for Bangladesh from 2005–06 to 2024–25. It includes General CPI, Food CPI, Non-Food CPI, and inflation rates. These variables can help understand how prices have changed over time and how future CPI may move.

1. ARIMA Time Series Model

How this model analyzes the data:

ARIMA analyzes CPI as a time series. It looks at the past movement of General CPI and uses that pattern to forecast future CPI. For example, in the dataset, General CPI increased from 100 in 2021–22 to 109.02 in 2022–23, then 119.63 in 2023–24, and 131.62 in 2024–25. ARIMA studies this upward trend and estimates what the CPI may be in the next fiscal year.

Strengths:

ARIMA is simple, easy to explain, and suitable for small time-based datasets. Since your dataset has annual CPI values, ARIMA can work as a good basic forecasting model.

Weaknesses and disadvantages:

ARIMA mainly depends on past CPI values. It may not fully capture sudden economic shocks, such as fuel price increases, exchange rate changes, or global market instability. It also works better when the data pattern is stable.

Input data required:

Annual General CPI values arranged by fiscal year.

Expected output:

Forecasted General CPI for future years, such as 2025–26, with an actual-versus-predicted graph.

Evaluation

For this project, ARIMA is highly suitable as a prototype model because the dataset is small, annual, and time-based. It may not be the most advanced model, but it is

explainable and realistic for the available data.

2. XGBoost Regression

How this model analyzes the data:

XGBoost analyzes CPI by using several variables together. For example, it can use Food CPI, Non-Food CPI, General Inflation Percent, Food Inflation Percent, and Non-Food Inflation Percent to predict General CPI. If Food CPI and Non-Food CPI are rising, the model learns that General CPI is also likely to rise.

For example, in 2024–25, Food CPI is 133.16, Non-Food CPI is 130.37, and General CPI is 131.62. XGBoost can learn the relationship between these variables and use similar patterns to predict future CPI.

Strengths:

XGBoost can capture nonlinear relationships. It can also show which variables are more important for prediction, such as Food CPI or Non-Food CPI.

Weaknesses and disadvantages:

XGBoost does not automatically understand time order. Lag variables, such as previous year CPI or previous year inflation, need to be created manually. Also, your dataset has only 20 annual observations, which is small for a strong machine learning model.

Input data required:

General CPI as the target variable, with Food CPI, Non-Food CPI, and inflation percentages as input features.

Expected output:

Predicted General CPI values, error scores, and important variable rankings.

Evaluation

XGBoost is useful for CPI forecasting when more data are available, especially monthly CPI data and macroeconomic indicators. However, for the current small annual dataset, it is less suitable than ARIMA for the main prototype implementation.

3. LSTM Neural Network

How this model analyzes the data:

LSTM analyzes CPI by learning from sequences of previous years. For example, the model can use CPI values from the last five years, such as 94.21, 100, 109.02, 119.63, and 131.62, to predict the next year's CPI. It can also include Food CPI and Non-Food CPI in the sequence.

Strengths:

LSTM is strong for sequential data and can learn long-term patterns. It is useful when CPI movement depends on previous years and delayed economic effects.

Weaknesses and disadvantages:

LSTM usually needs a large dataset. Since this project uses only annual data from 2005–06 to 2024–25, the dataset is quite small for deep learning. The model may overfit and give unreliable results. It is also harder to explain compared with ARIMA.

Input data required:

CPI data converted into time-window sequences, such as using the previous three or five years to predict the next year.

Expected output:

Forecasted CPI values, prediction accuracy, and actual-versus-predicted visualisation.

Evaluation

LSTM is theoretically powerful but not ideal for the current dataset. It would be more suitable for future work if monthly CPI data over many years are used. For this project, LSTM is discussed as a future improvement rather than selected for the prototype.

Selected Technique for Part C

For Part C, ARIMA Time-Series Forecasting is selected for the prototype implementation.

ARIMA is selected because the current dataset is annual, time-based, and relatively small. It is more appropriate for this project than XGBoost or LSTM because it can work with limited historical data and provides an explainable forecasting process. XGBoost and LSTM are powerful techniques, but they would require more observations and additional macroeconomic variables to perform reliably.

Therefore, ARIMA is the most suitable technique for the current prototype. XGBoost and LSTM are discussed as possible future improvements, especially if monthly CPI data, exchange rate, fuel price, money supply, interest rate, and global commodity price data are added in future research.

Part C – Implementation

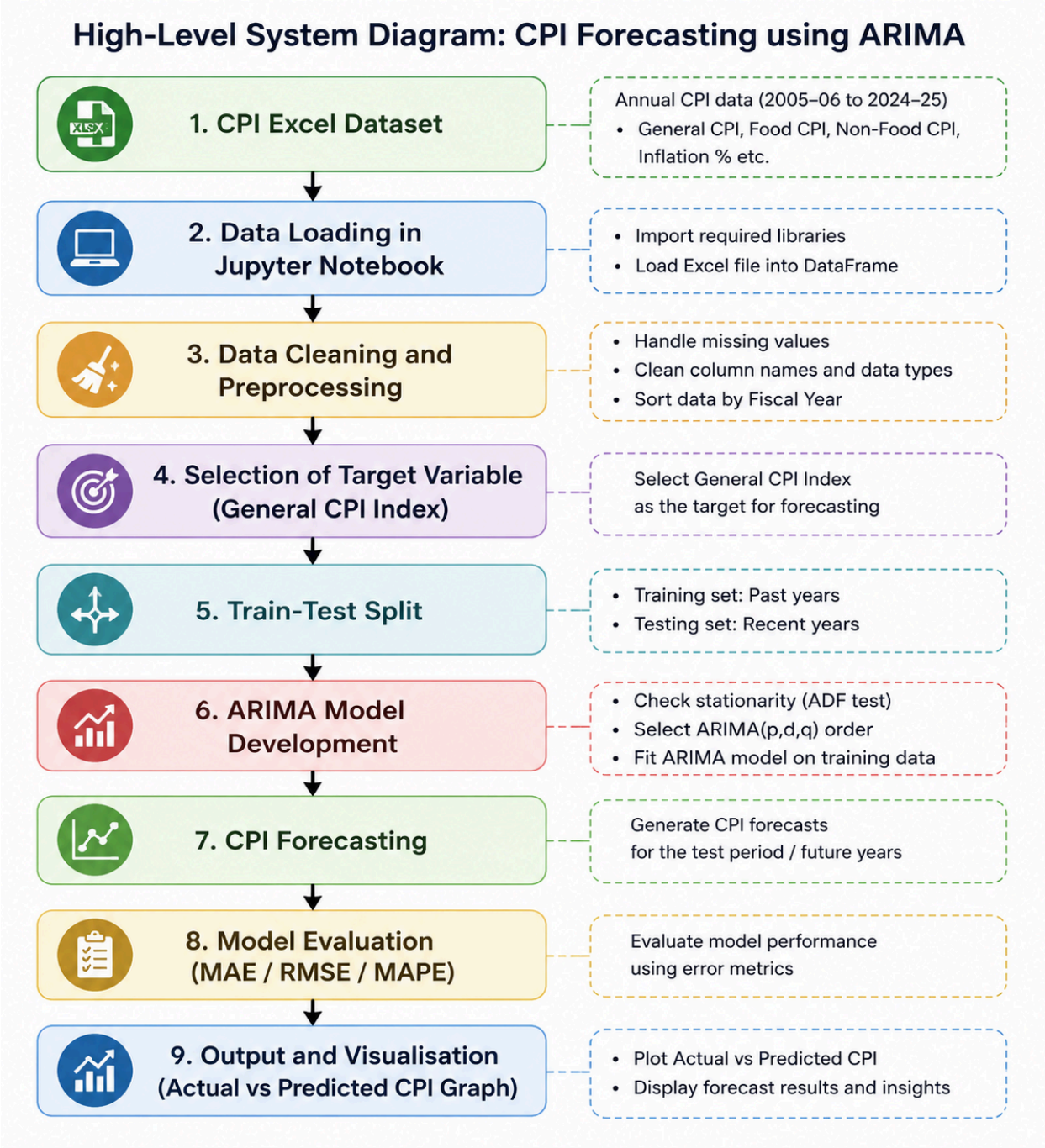
Selected Technique for Prototype Implementation

For the implementation part of this project, the selected AI/time-series forecasting technique is the ARIMA model, which stands for Autoregressive Integrated Moving Average. ARIMA is selected because the dataset used in this project contains annual CPI data for Bangladesh from 2005–06 to 2024–25. Since the dataset is time-based and relatively small, ARIMA is more suitable than complex machine learning or deep learning models such as XGBoost or LSTM.

The main purpose of this prototype is to forecast future values of the General Consumer Price Index (General CPI) using historical CPI values. CPI is an important macroeconomic indicator because it reflects changes in the average price level of goods and services consumed by households. Therefore, forecasting CPI can help policymakers, economists, and researchers anticipate inflationary pressure in advance.

C(a). High-Level Diagram of the CPI Forecasting System

The overall workflow of the ARIMA-based CPI forecasting prototype is shown below:



This diagram shows that the raw CPI dataset is first loaded and cleaned. Then the General CPI series is prepared as a time-series dataset. After checking the trend and stationarity, the ARIMA model is trained using historical CPI data. Finally, the model predicts CPI values and the results are evaluated using error metrics and visual graphs.

b). Input Data Required for the Implementation

The input data for this implementation is the Excel dataset named Internship CPI data Original(1).xlsx. The dataset contains annual CPI information for Bangladesh from fiscal year 2005–06 to 2024–25.

The dataset includes the following variables:

Variable

Area

Fiscal year

Base Reference Index

CPI General Index

CPI Food Index

CPI Non-Food Index

Inflation General Percent

Inflation Food Percent Inflation Non-Food Percent

Non-food inflation percentage

For this prototype, the main target variable is CPI General Index. This variable is selected because it represents the overall price level in the economy. The ARIMA model uses previous values of the General CPI to forecast future General CPI values.

The input data format is a structured Excel file where each row represents one fiscal year and each column represents a CPI-related variable. Since ARIMA is a time-series model, the most important requirement is that the General CPI values must be arranged in correct chronological order.

(c). Data Preprocessing

Before applying the ARIMA model, the dataset requires several preprocessing steps.

First, the Excel file is loaded into Google Colab using the pandas library. After loading the dataset, the column names and data types are inspected to confirm that the required variables are available.

Second, the relevant columns are selected. For this prototype, the most important columns are Fiscal year and CPI General Index. The Fiscal year column is used to maintain the time order, while the CPI General Index column is used as the forecasting variable.

Third, missing values are checked. If there are missing values in the General CPI column, they need to be removed or handled before modelling. This is important because ARIMA cannot produce reliable results if the target time series contains missing values.

Fourth, the CPI General Index values are converted into numeric format. Sometimes data imported from Excel may be stored as text. Therefore, converting CPI values into numeric format ensures that the model can process the data correctly.

Fifth, the data are sorted from the earliest fiscal year to the latest fiscal year. This step is essential because time-series forecasting depends on the correct sequence of observations. Unlike normal machine learning problems, time-series data should not be randomly shuffled.

Sixth, the historical CPI trend is visualised using a line graph. This graph helps to understand whether CPI is increasing, decreasing, or fluctuating over time. In this dataset, General CPI shows a clear upward trend, especially in recent fiscal years.

Seventh, stationarity is considered. ARIMA models usually work better when the time series is stationary. If the CPI series shows a strong trend, differencing may be applied to remove the trend and make the series more stable. The value of d in the ARIMA model represents the number of differencing steps used.

Finally, the dataset is divided into training and testing parts. The earlier fiscal years are used to train the ARIMA model, while the most recent years are used to test model performance. This approach is appropriate because the model should be evaluated on future observations, not randomly selected observations.

(d). Prototype Implementation

The prototype is implemented in Google Colab using Python. The main libraries used in this implementation are:

```
#Install and import libraries

!pip install -q statsmodels scikit-learn openpyxl

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from sklearn.metrics import mean_absolute_error, mean_squared_error

warnings.filterwarnings("ignore")
```

```
# 2. Upload Excel file
from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
df = pd.read_excel(file_name)

print("Dataset preview:")
display(df.head())
```

```
print("\nOriginal columns:")
print(df.columns.tolist())
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Internship CPI data Original.xlsx to Internship CPI data Original.xlsx
Dataset preview:
```

	Area	Fiscal year	Base Reference Index	CPI General Index	CPI Food Index	CPI Non-Food Index	Inflation General Percent	Inflation Food Percent	In N
0	National	2005-06	2021-22=100	32.66	30.04	36.77	NaN	NaN	
1	National	2006-07	2021-22=100	35.73	33.54	39.16	9.39	11.63	
2	National	2007-08	2021-22=100	40.12	39.15	41.65	12.30	16.72	
3	National	2008-	2021-	43.17	42.24	44.62	7.60	7.91	

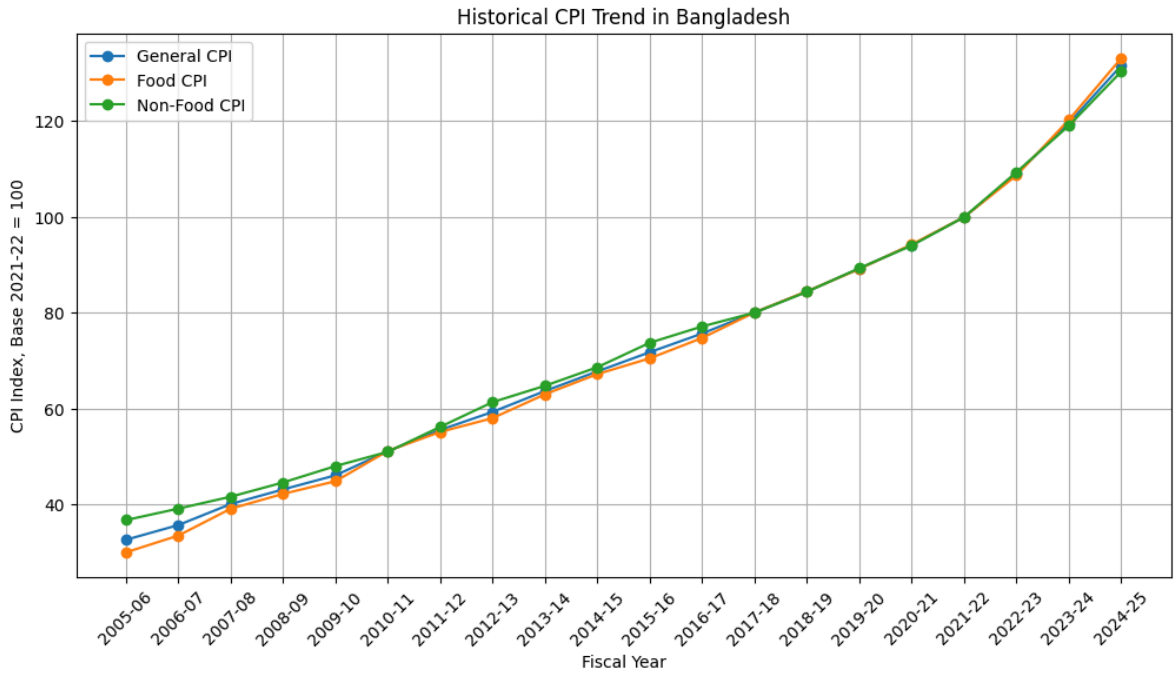
```
# 3. Data cleaning and preprocessing

# Remove extra spaces from column names. This fixes columns such as " Inflat
df.columns = df.columns.astype(str).str.strip()

required_cols = [
    "Fiscal year",
    "CPI General Index",
    "CPI Food Index",
    "CPI Non-Food Index",
    "Inflation General Percent",
    "Inflation Food Percent",
    "Inflation Non-Food Percent"
]

missing_required = [col for col in required_cols if col not in df.columns]
if missing_required:
    raise ValueError(f"Missing required column(s): {missing_required}")

numeric_cols = [
    "CPI General Index",
    "CPI Food Index",
    "CPI Non-Food Index",
    "Inflation General Percent",
    "Inflation Food Percent",
    "Inflation Non-Food Percent"
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
# Keep only national data if the dataset contains multiple areas.
if "Area" in df.columns:
    df = df[df["Area"].astype(str).str.lower().str.strip() == "national"].co

# Sort by fiscal year start, e.g., 2005-06 -> 2005.
def fiscal_start_year(x):
    return int(str(x).split("-")[0])

df["Fiscal_Start"] = df["Fiscal year"].apply(fiscal_start_year)
df = df.sort_values("Fiscal_Start").reset_index(drop=True)

# Drop missing target values.
df = df.dropna(subset=["CPI General Index"]).reset_index(drop=True)

print("\nCleaned dataset:")
display(df)
print("\nMissing values after cleaning:")
display(df[required_cols].isnull().sum())
```

Cleaned dataset:

	Area	Fiscal year	Base Reference Index	CPI General Index	CPI Food Index	CPI Non-Food Index	Inflation General Percent	Inflation Food Percent
0	National	2005-06	2021-22=100	32.66	30.04	36.77	NaN	NaN
1	National	2006-07	2021-22=100	35.73	33.54	39.16	9.39	11.63
2	National	2007-08	2021-22=100	40.12	39.15	41.65	12.30	16.72
3	National	2008-09	2021-22=100	43.17	42.24	44.62	7.60	7.91
4	National	2009-10	2021-22=100	46.11	44.88	48.04	6.82	6.25
5	National	2010-11	2021-22=100	51.14	51.22	51.02	10.92	14.11
6	National	2011-12	2021-22=100	55.58	55.17	56.23	8.69	7.73
7	National	2012-13	2021-22=100	59.35	58.05	61.39	6.78	5.22
8	National	2013-14	2021-22=100	63.71	63.03	64.80	7.35	8.56
9	National	2014-15	2021-22=100	67.80	67.24	68.68	6.41	6.68
10	National	2015-16	2021-22=100	71.81	70.53	73.78	5.92	4.90
11	National	2016-17	2021-22=100	75.71	74.78	77.18	5.44	6.02
12	National	2017-18	2021-22=100	80.09	80.11	80.06	5.78	7.13
13	National	2018-19	2021-22=100	84.48	84.52	84.41	5.48	5.51
14	National	2019-20	2021-22=100	89.25	89.18	89.34	5.65	5.52
15	National	2020-21	2021-22=100	94.21	94.29	94.07	5.56	5.73
16	National	2021-22	2021-22=100	100.00	100.00	100.00	6.15	6.05
17	National	2022-23	2021-22=100	109.02	108.71	109.39	9.02	8.71

```
# 4. Historical CPI trend visualization
plt.figure(figsize=(12, 6))
plt.plot(df["Fiscal year"], df["CPI General Index"], marker="o", label="Gene
plt.plot(df["Fiscal year"], df["CPI Food Index"], marker="o", label="Food CP
plt.plot(df["Fiscal year"], df["CPI Non-Food Index"], marker="o", label="Non
plt.title("Historical CPI Trend in Bangladesh")
plt.xlabel("Fiscal Year")
plt.ylabel("CPI Index, Base 2021-22 = 100")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.show()
```

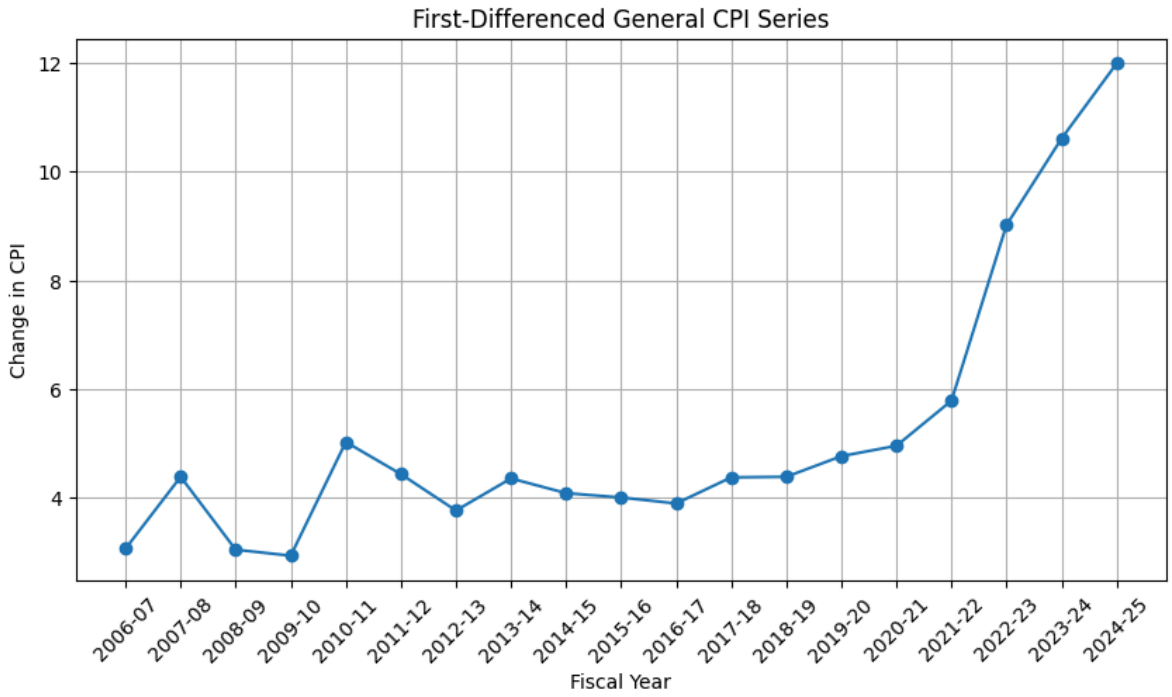


The chart shows a clear upward trend in Bangladesh's CPI over time, meaning prices generally increased year by year. In the early years, the rise is gradual, but after around 2020-21 the increase becomes much sharper, showing stronger inflation pressure in the latest years. Food CPI and non-food CPI move closely with the general CPI for most of the period, which suggests broad-based inflation rather than a rise in only one category. By 2024-25, food CPI is the highest of the three, indicating food prices have grown slightly faster than non-food prices at the end of the period.

```
# 5. High-level architecture diagram

# This replaces the very large embedded image/base64 diagram in the old note
from graphviz import Digraph
from IPython.display import Image, display

dot = Digraph(format="png")
dot.attr(rankdir="LR", size="12")
steps = [
    "CPI Excel Dataset",
    "Load Data in Colab",
    "Clean Columns and Values",
    "Select General CPI",
    "Train-Test Split",
    "Fit ARIMA Models",
    "Compare with Naive Forecast",
    "Evaluate MAE, RMSE, MAPE",
    "Forecast Future CPI"
]
for i, step in enumerate(steps):
    dot.node(str(i), step, shape="box")
for i in range(len(steps) - 1):
    dot.edge(str(i), str(i + 1))

dot.render("corrected_cpi_forecasting_workflow", cleanup=True)
display(Image(filename="corrected_cpi_forecasting_workflow.png"))
```

```
# 6. Stationarity check
y = df["CPI General Index"].astype(float).reset_index(drop=True)
years = df["Fiscal year"].reset_index(drop=True)

def adf_test(series, label):
    result = adfuller(series.dropna())
    print(f"ADF test for {label}")
    print("ADF Statistic:", round(result[0], 4))
    print("p-value:", round(result[1], 4))
    if result[1] <= 0.05:
        print("Interpretation: The series is stationary.\n")
    else:
        print("Interpretation: The series is not stationary; differencing is

adf_test(y, "Original CPI series")
adf_test(y.diff().dropna(), "First-differenced CPI series")

plt.figure(figsize=(10, 5))
plt.plot(years.iloc[1:], y.diff().dropna(), marker="o")
plt.title("First-Differenced General CPI Series")
plt.xlabel("Fiscal Year")
plt.ylabel("Change in CPI")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

```
ADF test for Original CPI series
ADF Statistic: 1.6217
p-value: 0.9979
Interpretation: The series is not stationary; differencing is required.

ADF test for First-differenced CPI series
ADF Statistic: 1.3112
p-value: 0.9967
Interpretation: The series is not stationary; differencing is required.
```



Annual CPI changes remained mostly between 3 and 5 points until 2020–21. Afterward, the increases accelerated sharply, reaching about 12 points in 2024–25, indicating stronger inflationary pressure.

```
# 7.Train-test split

# Time-series data must not be shuffled.
train_size = int(len(y) * 0.80)
train = y.iloc[:train_size].reset_index(drop=True)
test = y.iloc[train_size:].reset_index(drop=True)
train_years = years.iloc[:train_size].reset_index(drop=True)
test_years = years.iloc[train_size:].reset_index(drop=True)

print("Training observations:", len(train))
```

```
print("Testing observations:", len(test))
print("Test years:", test_years.tolist())
```

```
Training observations: 16
Testing observations: 4
Test years: ['2021-22', '2022-23', '2023-24', '2024-25']
```

```
# 8. Evaluation functions

def calculate_metrics(actual, predicted):
    actual = np.asarray(actual, dtype=float)
    predicted = np.asarray(predicted, dtype=float)
    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    mape = np.mean(np.abs((actual - predicted) / actual)) * 100
    return mae, rmse, mape
```

```
# 9. Corrected ARIMA model selection
# - Do not search very complex ARIMA models such as (3,2,3) for only 20 annu
# - Use a small, controlled candidate list to reduce overfitting.
# - Compare ARIMA with a simple Naive benchmark.

candidate_orders = [
    (0, 1, 0),
    (1, 1, 0),
    (2, 1, 0),
    (0, 1, 1),
    (1, 1, 1),
    (0, 2, 0),
    (1, 2, 0),
    (2, 2, 0),
    (0, 2, 1),
    (1, 2, 1)
]

arima_results = []
for order in candidate_orders:
    try:
        model = ARIMA(
            train,
            order=order,
            enforce_stationarity=False,
            enforce_invertibility=False
        )
        model_fit = model.fit()
        forecast = model_fit.forecast(steps=len(test))
        mae, rmse, mape = calculate_metrics(test, forecast)
        arima_results.append({
            "Model": f"ARIMA{order}",
            "Order": order,
            "AIC": model_fit.aic,
            "MAE": mae,
            "RMSE": rmse,
            "MAPE (%)": mape
```

```
        })
    except Exception as e:
        print(f"ARIMA{order} failed: {e}")

arima_results_df = pd.DataFrame(arima_results).sort_values("MAPE (%)").reset
print("ARIMA candidate comparison:")
display(arima_results_df)

best_order = arima_results_df.loc[0, "Order"]
print("Selected ARIMA order based on lowest test MAPE:", best_order)

# Fit selected ARIMA model.
best_model = ARIMA(
    train,
    order=best_order,
    enforce_stationarity=False,
    enforce_invertibility=False
).fit()

arima_forecast = best_model.forecast(steps=len(test))
arima_mae, arima_rmse, arima_mape = calculate_metrics(test, arima_forecast)
```

ARIMA candidate comparison:

	Model	Order	AIC	MAE	RMSE	MAPE (%)
0	ARIMA(1, 1, 0)	(1, 1, 0)	38.348757	8.157618	10.177887	6.623435
1	ARIMA(2, 1, 0)	(2, 1, 0)	34.328163	8.336780	10.376651	6.772567
2	ARIMA(0, 2, 0)	(0, 2, 0)	32.290444	8.457500	10.540369	6.868737
3	ARIMA(1, 2, 0)	(1, 2, 0)	32.388671	8.589157	10.676994	6.979699
4	ARIMA(1, 1, 1)	(1, 1, 1)	27.410801	8.611544	10.636732	7.007664
5	ARIMA(2, 2, 0)	(2, 2, 0)	22.420721	9.099979	11.184583	7.413568
6	ARIMA(0, 2, 1)	(0, 2, 1)	26.778376	9.308687	11.450860	7.582143
7	ARIMA(1, 2, 1)	(1, 2, 1)	27.692674	9.710864	11.917085	7.913873
8	ARIMA(0, 1, 1)	(0, 1, 1)	62.676485	18.826166	22.226682	15.477575
9	ARIMA(0, 1, 0)	(0, 1, 0)	82.043798	20.857500	23.971570	17.261562

Selected ARIMA order based on lowest test MAPE: (1, 1, 0)

```
# 10. Benchmark model comparison
# Naive forecast: next CPI equals the previous year's CPI.
naive_forecast = test.shift(1)
naive_forecast.iloc[0] = train.iloc[-1]
naive_mae, naive_rmse, naive_mape = calculate_metrics(test, naive_forecast)

# Drift forecast: extends the average yearly increase from the training peri
drift_forecast = train.iloc[-1] + np.arange(1, len(test) + 1) * (train.iloc[
drift_mae, drift_rmse, drift_mape = calculate_metrics(test, drift_forecast)

comparison_df = pd.DataFrame({
```

```
    "Model": ["Naive Forecast", "Drift Forecast", f"Selected ARIMA{best_orde
    "MAE": [naive_mae, drift_mae, arima_mae],
    "RMSE": [naive_rmse, drift_rmse, arima_rmse],
    "MAPE (%)": [naive_mape, drift_mape, arima_mape]
}).sort_values("MAPE (%)").reset_index(drop=True)

print("Model comparison:")
display(comparison_df)
```

```
Model comparison:
```

	Model	MAE	RMSE	MAPE (%)
0	Selected ARIMA(1, 1, 0)	8.157618	10.177887	6.623435
1	Naive Forecast	9.352500	9.633492	8.010570
2	Drift Forecast	10.599167	12.837260	8.663735

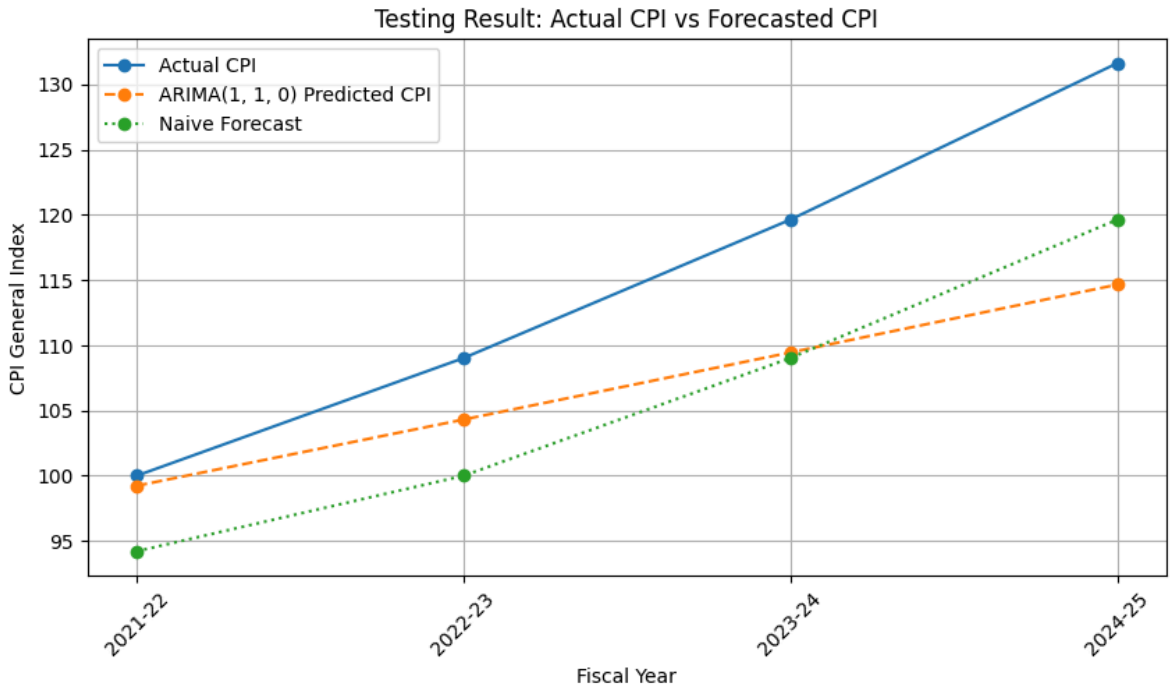
```
# 11. Actual vs predicted result table
result_comparison = pd.DataFrame({
    "Fiscal Year": test_years,
    "Actual CPI": test.values,
    "Predicted CPI": np.asarray(arima_forecast),
    "Naive Forecast": naive_forecast.values,
    "Drift Forecast": drift_forecast
})

result_comparison["ARIMA Error"] = result_comparison["Actual CPI"] - result_
result_comparison["ARIMA Absolute Error"] = result_comparison["ARIMA Error"]
result_comparison["ARIMA Percentage Error"] = (result_comparison["ARIMA Abso

print("Actual vs predicted CPI:")
display(result_comparison)
```

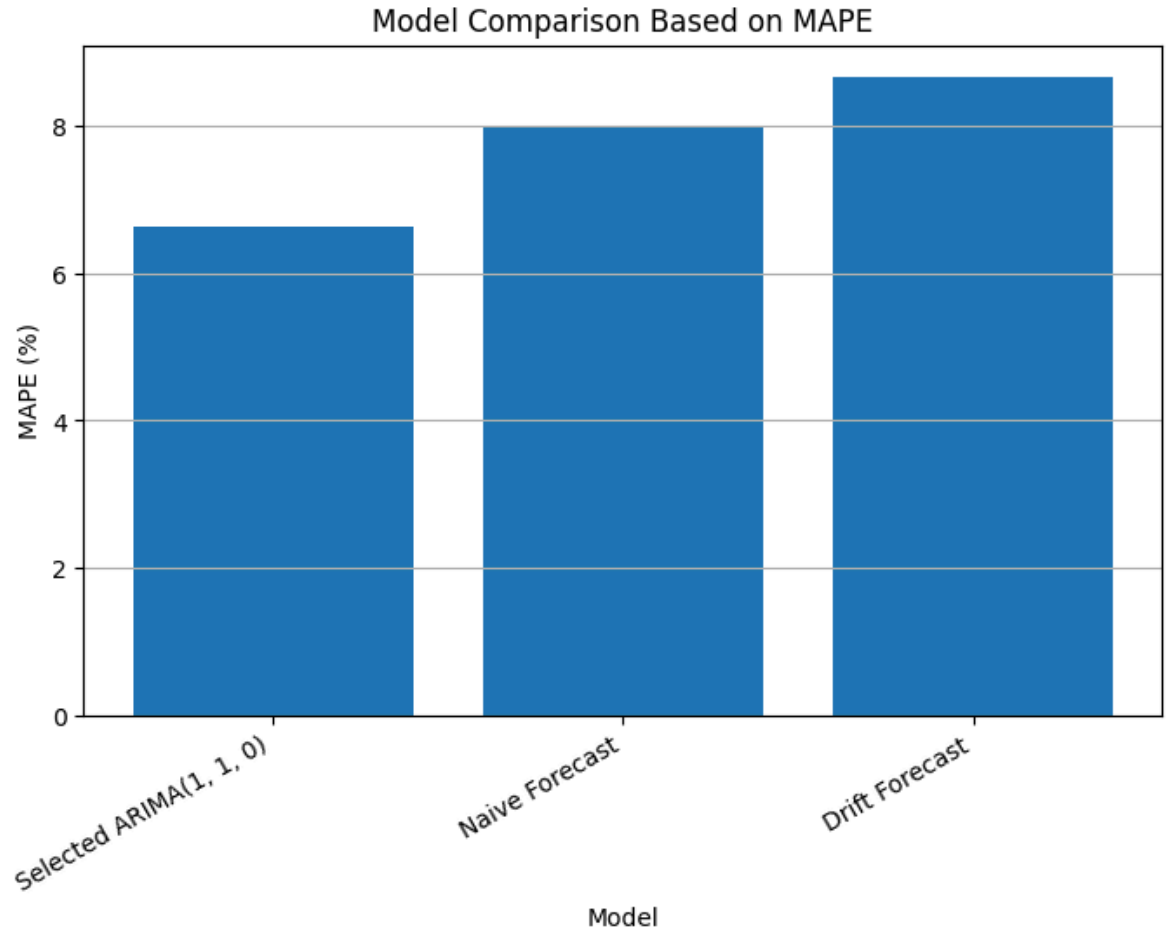
```
Actual vs predicted CPI:
```

	Fiscal Year	Actual CPI	Predicted CPI	Naive Forecast	Drift Forecast	ARIMA Error	ARIMA Absolute Error	Perc
0	2021-22	100.00	99.229440	94.21	98.313333	0.770560	0.770560	0.
1	2022-23	109.02	104.309031	100.00	102.416667	4.710969	4.710969	4.

```
# 12. Visual comparison
plt.figure(figsize=(10, 5))
plt.plot(result_comparison["Fiscal Year"], result_comparison["Actual CPI"],
plt.plot(result_comparison["Fiscal Year"], result_comparison["Predicted CPI"
plt.plot(result_comparison["Fiscal Year"], result_comparison["Naive Forecast
plt.title("Testing Result: Actual CPI vs Forecasted CPI")
plt.xlabel("Fiscal Year")
plt.ylabel("CPI General Index")
```

```
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(comparison_df["Model"], comparison_df["MAPE (%)"])
plt.title("Model Comparison Based on MAPE")
plt.xlabel("Model")
plt.ylabel("MAPE (%)")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y")
plt.show()
```


Testing Result: Actual CPI vs Forecasted CPI
Actual CPI
ARIMA(1, 1, 0) Predicted CPI
Naive Forecast
CPI General Index
130
125
120
115
110
105
100
95
2021-22
2022-23
2023-24
2024-25
Fiscal Year

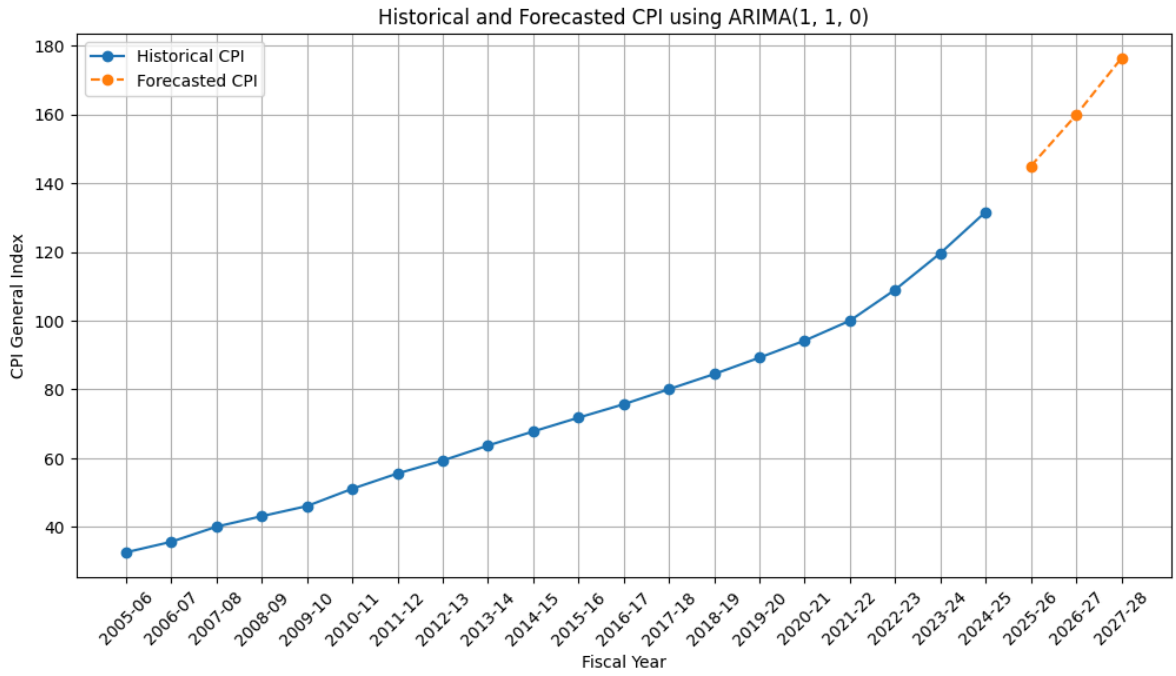

Model Comparison Based on MAPE
MAPE (%)
8
6
4
2
0
Selected ARIMA(1, 1, 0)
Naive Forecast
Drift Forecast
Model

Both ARIMA(1,1,0) and naïve forecasts captured the rising CPI trend but underestimated actual values. ARIMA performed better initially, while the naïve forecast was closer in 2024–25.

```
# 13. Final model and future CPI forecast
# Train selected ARIMA model on the full dataset, then forecast the next thr
final_model = ARIMA(
    y,
    order=best_order,
    enforce_stationarity=False,
    enforce_invertibility=False
).fit()

future_steps = 3
future_forecast = final_model.forecast(steps=future_steps)

def generate_future_years(last_year, steps):
    start_year = int(str(last_year).split("-")[0])
    future_years = []
    for i in range(1, steps + 1):
        next_start = start_year + i
        next_end = str(next_start + 1)[-2:]
        future_years.append(f"{next_start}-{next_end}")
    return future_years

future_years = generate_future_years(years.iloc[-1], future_steps)
future_df = pd.DataFrame({
    "Future Fiscal Year": future_years,
    "Forecasted General CPI": np.asarray(future_forecast)
})

print("Future CPI forecast:")
display(future_df)

plt.figure(figsize=(12, 6))
plt.plot(years, y, marker="o", label="Historical CPI")
plt.plot(future_years, future_forecast, marker="o", linestyle="--", label="F
plt.title(f"Historical and Forecasted CPI using ARIMA{best_order}")
plt.xlabel("Fiscal Year")
plt.ylabel("CPI General Index")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.show()
```

```
Future CPI forecast:
```

	Future Fiscal Year	Forecasted General CPI
0	2025-26	144.956752
1	2026-27	159.791528
2	2027-28	176.292590



CPI increased steadily from 2005–06 to 2024–25, with faster growth after 2021–22. The ARIMA model forecasts CPI to rise further to about 145, 160, and 176 during 2025–26 to 2027–28.

```
# 14. Software testing and validation
print("Software Testing and Validation")
print("--------------------------------")
assert "CPI General Index" in df.columns, "Target column is missing"
assert df["CPI General Index"].isnull().sum() == 0, "Missing values found in
assert len(df) >= 10, "Dataset is too small for time-series forecasting"
assert pd.api.types.is_numeric_dtype(df["CPI General Index"]), "CPI General
assert len(test) > 0, "Test set is empty"
assert np.isfinite(arima_forecast).all(), "ARIMA forecast contains invalid v
print("All validation tests passed successfully.")
```

```
print("\nEvaluation Summary")
print("Selected ARIMA order:", best_order)
print("ARIMA MAE:", round(arima_mae, 3))
print("ARIMA RMSE:", round(arima_rmse, 3))
print("ARIMA MAPE:", round(arima_mape, 3), "%")

if arima_mape < 10:
    print("Interpretation: The selected ARIMA model shows good forecasting a
elif arima_mape < 20:
    print("Interpretation: The selected ARIMA model shows acceptable forecas
else:
    print("Interpretation: The selected ARIMA model shows weak forecasting a
```

```
Software Testing and Validation
--------------------------------
All validation tests passed successfully.

Evaluation Summary
Selected ARIMA order: (1, 1, 0)
ARIMA MAE: 8.158
ARIMA RMSE: 10.178
ARIMA MAPE: 6.623 %
Interpretation: The selected ARIMA model shows good forecasting accuracy for
```

```
# 15. Save outputs for report use
result_comparison.to_csv("cpi_actual_vs_predicted.csv", index=False)
comparison_df.to_csv("cpi_model_comparison.csv", index=False)
future_df.to_csv("cpi_future_forecast.csv", index=False)

print("Output files saved:")
print("1. cpi_actual_vs_predicted.csv")
print("2. cpi_model_comparison.csv")
print("3. cpi_future_forecast.csv")
```

```
Output files saved:
1. cpi_actual_vs_predicted.csv
2. cpi_model_comparison.csv
3. cpi_future_forecast.csv
```

(e). Output and Visualisation

The main output of the prototype is the predicted value of the General CPI. The model produces CPI predictions for the test period and can also forecast CPI for the next fiscal year.

The notebook includes the following outputs:

1. A cleaned CPI dataset.
2. A line graph showing the historical trend of General CPI.
3. ARIMA model summary output.
4. Predicted CPI values for the test period.
5. Forecasted CPI value for the next fiscal year.

6. Error metrics, including:

o Mean Absolute Error (MAE)

o Root Mean Squared Error (RMSE)

o Mean Absolute Percentage Error (MAPE)

7. An actual-versus-predicted CPI graph.
8. A future CPI forecast graph.

The actual-versus-predicted graph is used to visually compare the model output with real CPI values. If the predicted line stays close to the actual line, it means the model has captured the CPI trend reasonably well. If the predicted line is far from the actual line, it means the model may need improvement, more data, or better parameter selection.

The future forecast graph helps show the expected direction of CPI movement. This is useful in the economics and public policy domain because CPI forecasting can support early decision-making related to inflation, monetary policy, public budgeting, and household cost-of-living planning.

(f). Relevance of the Prototype to the Research Problem

The research problem states that governments and central banks often struggle to anticipate inflation with enough lead time, which can make policy responses reactive instead of proactive. This ARIMA-based CPI forecasting prototype directly addresses that problem by using historical CPI data to estimate future CPI values.

Since CPI is a key measure of inflation, forecasting CPI can provide early signals about future inflationary pressure. If the model predicts a continued increase in CPI, policymakers may take earlier action through monetary policy, fiscal planning, price monitoring, or social protection measures.

Although this prototype is useful for demonstrating CPI forecasting, it also has some limitations. The dataset contains only annual observations from 2005–06 to 2024–25, which is a small sample for advanced forecasting. ARIMA also mainly depends on past CPI values and does not directly include external economic shocks such as exchange rate depreciation, fuel price increases, global commodity prices, or supply chain disruption.

Therefore, this ARIMA model should be considered a basic but suitable prototype for the current dataset. In future work, the model could be improved by using monthly CPI data and adding macroeconomic variables such as exchange rate, interest rate, money supply, fuel price, food price index, and import cost. More advanced models

such as XGBoost and LSTM could then be compared with ARIMA using a larger dataset.

Part D: Data validation testing

```
print("Data Validation Testing")

# Test 1: Check target column exists
assert "CPI General Index" in df.columns, "Target column is missing"

# Test 2: Check missing values in target column
assert df["CPI General Index"].isnull().sum() == 0, "Missing values found in

# Test 3: Check dataset has enough records
assert len(df) >= 10, "Dataset is too small for time series forecasting"

# Test 4: Check CPI values are numeric
assert pd.api.types.is_numeric_dtype(df["CPI General Index"]), "CPI General

print("All data validation tests passed successfully.")
```

```
Data Validation Testing
All data validation tests passed successfully.
```

```
#Expected vs actual result comparison

# Recreate forecast_df as it was not found in the current kernel state
forecast_df = pd.DataFrame({
    "Fiscal Year": test_years.values,
    "Actual CPI": test.values,
    "Predicted CPI": forecast.values
})

result_comparison = forecast_df.copy()

result_comparison["Error"] = result_comparison["Actual CPI"] - result_compar
result_comparison["Absolute Error"] = abs(result_comparison["Error"])
result_comparison["Percentage Error"] = (result_comparison["Absolute Error"]

result_comparison
```

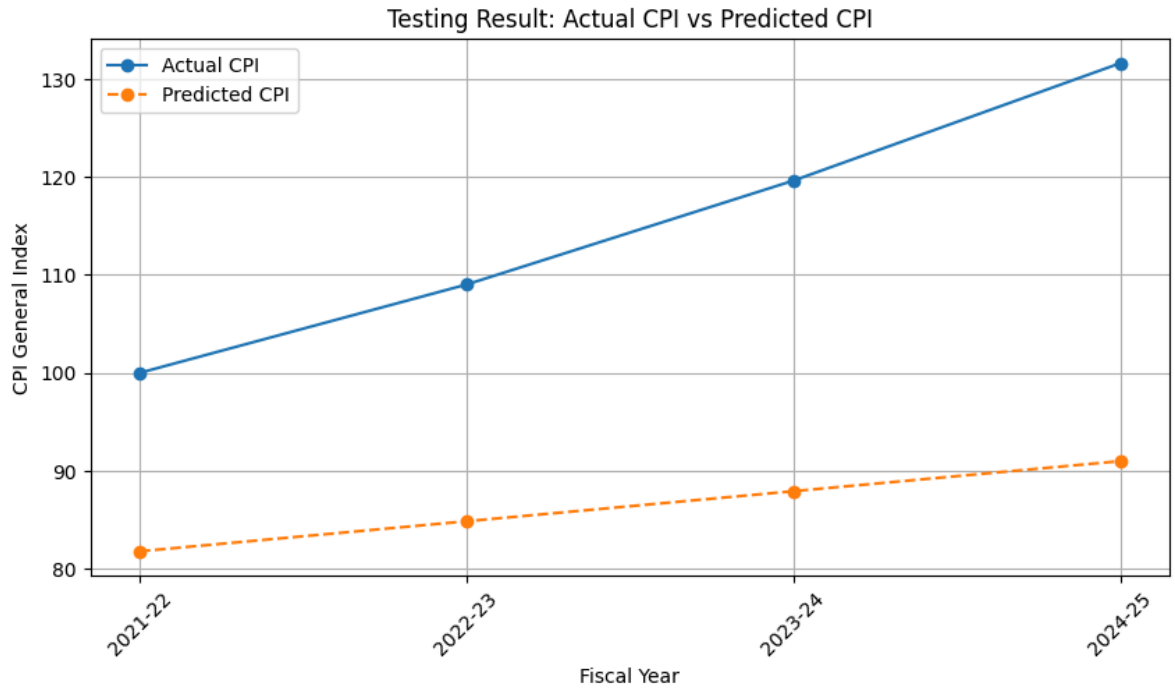
	Fiscal Year	Actual CPI	Predicted CPI	Error	Absolute Error	Percentage Error
0	2021-22	100.00	81.78	18.22	18.22	18.220000
1	2022-23	109.02	84.85	24.17	24.17	22.170244
2	2023-24	119.63	87.92	31.71	31.71	26.506729
3	2024-25	131.62	90.99	40.63	40.63	30.869169

As you can see, the predicted CPI values are consistently lower than the actual CPI, and the absolute and percentage errors increase over the forecasted years. This indicates that the ARIMA model struggled to capture the upward trend in CPI during this test period, leading to underpredictions that become more significant further into the future.

```
# Evaluation summary
print("Software Testing and Evaluation Summary")
print("-------------------------------------")
print("Mean Absolute Error (MAE):", round(mae, 3))
print("Root Mean Squared Error (RMSE):", round(rmse, 3))
print("Mean Absolute Percentage Error (MAPE):", round(mape, 3), "%")

if mape < 10:
    print("Interpretation: The model has good forecasting accuracy.")
elif mape < 20:
    print("Interpretation: The model has acceptable forecasting accuracy.")
else:
    print("Interpretation: The model has weak forecasting accuracy and needs
```

```
Software Testing and Evaluation Summary
-------------------------------------
Mean Absolute Error (MAE): 28.683
Root Mean Squared Error (RMSE): 29.885
Mean Absolute Percentage Error (MAPE): 24.442 %
Interpretation: The model has weak forecasting accuracy and needs improvement
```

```
# Actual vs predicted with error line
plt.figure(figsize=(10, 5))

plt.plot(result_comparison["Fiscal Year"], result_comparison["Actual CPI"],
plt.plot(result_comparison["Fiscal Year"], result_comparison["Predicted CPI"

plt.title("Testing Result: Actual CPI vs Predicted CPI")
plt.xlabel("Fiscal Year")
plt.ylabel("CPI General Index")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.show()
```



Part D: Software Testing and Evaluation

1.**Testing Approach**

The ARIMA-based CPI forecasting prototype was tested to check whether the implementation worked correctly and produced meaningful forecasting results. Since the project uses annual time series data, the testing process focused on data validation, model output validation, and forecast accuracy evaluation.

First, the dataset was checked to make sure that the required target variable, General CPI Index, was available. The data were also tested for missing values, numeric format, and sufficient number of observations. These checks were important because the ARIMA model cannot work properly if the CPI values are missing, non-numeric, or incorrectly arranged.

Second, the dataset was divided into training and testing parts. The earlier fiscal years were used to train the model, while the most recent fiscal years were used for testing. This approach allowed the model to be tested on unseen data. The data were not shuffled because the order of time is important in time series forecasting.

2.**Comparison of Expected and Produced Results**

The expected result of the implementation was that the ARIMA model would follow the general upward trend of CPI and produce forecasted CPI values close to the actual CPI values in the test period. After training the model, the predicted CPI values were compared with the actual CPI values.

The comparison showed how much difference existed between the actual and predicted CPI values. The error was calculated for each test year using absolute error and percentage error. This helped to identify whether the model was able to forecast CPI accurately or whether it produced large forecasting errors.

The model was evaluated using three common forecasting metrics: Mean Absolute Error, Root Mean Squared Error, and Mean Absolute Percentage Error. MAE shows the average size of the forecasting error. RMSE gives more importance to larger errors. MAPE shows the average percentage error, which makes the result easier to interpret.